

# Técnicas Digitales 2 - Trabajo práctico Nro 2

*Braida Agustin (404621) - Ernst Pedro (400624) - Polizze Tomas (400793) - Valentino Rao (402308)*

*Técnicas Digitales 2 4R2, Ingeniería Electrónica, Facultad Regional Córdoba  
Universidad Tecnológica Nacional*

## I. INTRODUCCIÓN

El presente trabajo práctico aborda la programación de sistemas embebidos en nivel *bare metal*. El objetivo principal es comprender la interacción directa con los registros del hardware, prescindiendo de capas de abstracción para analizar el comportamiento del microcontrolador desde el encendido hasta la ejecución de la primera instrucción.

Para el desarrollo, se utiliza una placa **BluePill** con el microcontrolador STM32F103C8T6 (núcleo ARM Cortex-M3) y un programador **ST-Link V2**. El flujo de trabajo se basa íntegramente en el ecosistema **GNU ARM Tool-chain** bajo Linux, empleando herramientas estándar como `arm-none-eabi-gcc` para la compilación cruzada, `make` para la automatización, y `OpenOCD` para la grabación y depuración de la aplicación.

## II. BLINK LED SIN DELAY

El desarrollo del proyecto se llevó a cabo siguiendo los pasos detallados a continuación:

### A. Aplicación (main.c)

```
1 #include <stdint.h>
2
3 // register address
4 #define RCC_BASE      0x40021000
5 #define GPIOC_BASE    0x40011000
6 #define GPIOA_BASE    0x40010800
7
8 #define RCC_APB2ENR   *(volatile uint32_t *)
9                      (RCC_BASE + 0x18)
10 #define GPIOC_CRH     *(volatile uint32_t *)
11                      (GPIOC_BASE + 0x04)
12 #define GPIOA_CRL     *(volatile uint32_t *)
13                      (GPIOA_BASE + 0x00)
14 #define GPIOC_ODR     *(volatile uint32_t *)
15                      (GPIOC_BASE + 0x0C)
16 #define GPIOA_ODR     *(volatile uint32_t *)
17                      (GPIOA_BASE + 0x0C)
18
19 // bit fields
20 #define RCC_IOPCEN    (1 << 4)
21 #define RCC_IOPAEN    (1 << 2)
22
23 #define GPIOC13       (1UL << 13)
24 #define GPIOA0        (1UL << 0)
```

```
21 void main(void)
22 {
23     // Habilitacion de relojes para puerto
24     // A y C
25     RCC_APB2ENR |= RCC_IOPCEN;
26     RCC_APB2ENR |= RCC_IOPAEN;
27
28     // Configuracion PC13 como salida push-
29     // pull 2MHz
30     GPIOC_CRH &= 0xFF0FFFFFFF;
31     GPIOC_CRH |= 0x00200000;
32
33     // Configuracion PA0 como salida push-
34     // pull 2MHz
35     GPIOA_CRL &= 0xFFFFFFF0;
36     GPIOA_CRL |= 0x00000002;
37
38     while (1)
39     {
40         // Control LED en PC13
41         GPIOC_ODR |= GPIOC13;
42         for (int i = 0; i < 500000; i++);
43         GPIOC_ODR &= ~GPIOC13;
44         for (int i = 0; i < 50000; i++);
45
46         // Control LED en PA0
47         GPIOA_ODR |= GPIOA0;
48         for (int i = 0; i < 500000; i++);
49         GPIOA_ODR &= ~GPIOA0;
50         for (int i = 0; i < 50000; i++);
51     }
52 }
```

Listing 1. Implementacion de parpadeo de LEDs en main.c

### B. Análisis de Cambios

- **Gestión de Tiempos (SysTick vs. Bucle for):** El proyecto base utilizaba retardos arbitrarios mediante bucles `for`, los cuales dependen de la velocidad del procesador y no son precisos. La versión final implementa el temporizador de núcleo **SysTick** para generar interrupciones cada 1 ms. Esto permite una temporización exacta y determinística basada en el reloj del sistema.
- **Lógica de Ejecución:** Mientras que el original parpadeaba únicamente el LED integrado en la placa, la versión final alterna el estado de dos LED. Además, la función `delay_ms_bloqueante` utiliza una variable global `tick` incrementada por hardware, lo que representa

una transición hacia un manejo de eventos basado en interrupciones.

### C. Startup(crt.s)

```

1  .syntax unified
2  .cpu cortex-m3
3  .thumb
4  .extern _esstack
5
6  .section .text.manejadores // definimos la
    seccion text.manejadores
7
8  // funcion basica para manejar los punteros
9  .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b .           /* Bucle infinito */
13
14 // definir los vectores ejemplo
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak SysTick_Handler
18 .thumb_set SysTick_Handler, Default_Handler
19 .weak HardFault_Handler
20 .thumb_set HardFault_Handler,
    Default_Handler
21
22 // vectores
23 .section .isr_vector,"a",%progbits
24 .word _esstack
25 .word _reset
26 .word NMI_Handler           /* 2:  NMI (Non-
    Maskable Interrupt) */
27 .word HardFault_Handler    /* 3:  HardFault
    (falla grave) */
28 .word 0                    /* 4:  MemManage
    (falla de memoria) */
29 .word 0                    /* 5:  BusFault (
    falla de bus) */
30 .word 0                    /* 6:  UsageFault
    (falla de uso) */
31 .word 0                    /* 7:  Reservado
    */
32 .word 0                    /* 8:  Reservado
    */
33 .word 0                    /* 9:  Reservado
    */
34 .word 0                    /* 10: Reservado
    */
35 .word 0                    /* 11: SVCALL (
    Llamada al Supervisor) */
36 .word 0                    /* 12: Debug
    Monitor */
37 .word 0                    /* 13: Reservado
    */
38 .word 0                    /* 14: PendSV (
    Interrupcion pendiente del sistema) */
39 .word SysTick_Handler      /* 15: El handler
    del SysTick */
40
41 // declaramos la funcion _reset dentro de
    la seccion text.reset
42 .section .text.reset
43 .thumb_func
44 .global _reset
45 _reset:
46     bl main
47     b .

```

## • Ampliación de Periféricos GPIO:

### – Evolución de la Temporización:

- \* *Original:* Utilizaba bucles `for` vacíos. Este método es impreciso ya que el tiempo de retardo varía según la optimización del compilador y la frecuencia del reloj.
- \* *Modificado:* Implementa el **SysTick Timer** del núcleo Cortex-M3. Se configuró para generar una interrupción exacta cada 1 ms, permitiendo retardos determinísticos y reales.

### – Flujo de Ejecución e Interrupciones:

- \* *Original:* El programa era estrictamente secuencial y lineal.
- \* *Modificado:* Se introdujo el uso de interrupciones de hardware. El procesador suspende la tarea principal para ejecutar el `SysTick_Handler`, lo que representa un nivel de programación más avanzado y eficiente.

### D. LinkerScript (linker.ld)

```

1  MEMORY
2  {
3      FLASH (rx) : ORIGIN = 0x08000000, LENGTH
        = 64K
4      RAM (xrw)  : ORIGIN = 0x20000000, LENGTH
        = 20K
5  }
6
7  /* 2. Definicion de Simbolos Globales */
8  ENTRY(_reset) /* El punto de entrada,
    definido en tu crt.s */
9
10 /* Simbolo para el final de la pila, usado
    en tu .isr_vector */
11 _esstack = ORIGIN(RAM) + LENGTH(RAM); /* 0
    x20005000 (el final de los 20K de RAM)
    */
12
13 /* 3. Definicion de las Secciones de Salida
    */
14 SECTIONS
15 {
16     /* SECCIONES EN FLASH */
17     /* La tabla de vectores debe ir primero
        en la Flash */
18     .isr_vector :
19     {
20         KEEP(*( .isr_vector)) /* Mantiene la
            seccion .isr_vector de tu crt.s */
21     } > FLASH
22
23     /*Codigo del programa y datos de solo
        lectura */
24     .text :
25     {
26         *(.text)                /* Todo el codigo
            C y ensamblador */
27         *(.text*)               /* Variantes de .
            text */
28         *(.rodata)              /* Datos de solo
            lectura (constantes) */
29         *(.rodata*)
30     } > FLASH

```

```

31  /* SECCIONES EN RAM */
32  /* .data: Variables inicializadas con un
   valor distinto de cero (ej: int x =
   10;) */
33  /* El linker necesita simbolos para que
   el startup mueva los datos */
34  _sidata = LOADADDR(.data); /* Direccion
   de los valores en FLASH */
35
36  .data :
37  {
38      _sdata = .;          /* Inicio de .
   data en RAM */
39      *(.data)
40      *(.data*)
41      _edata = .;          /* Fin de .data
   en RAM */
42  } > RAM AT> FLASH      /* Ubicada en RAM,
   pero cargada desde FLASH */
43  /* .bss: Variables inicializadas a cero o
   sin inicializar */
44  /* (ej: static int y; o int z = 0;) */
45  .bss :
46  {
47      _sbss = .;           /* Simbolo de
   inicio de .bss */
48      *(.bss)
49      *(.bss*)
50      *(COMMON)
51      _ebss = .;           /* Simbolo de fin
   de .bss */
52  } > RAM                /* Solo ocupa
   espacio en RAM */
53  }

```

### E. Makefile

```

1  @@ -0,0 +1,38 @@
2  SRC_DIR = ./
3  BUILD_DIR = build
4  CC = arm-none-eabi-gcc
5  AS = arm-none-eabi-as
6  LD = arm-none-eabi-ld
7  BIN = arm-none-eabi-objcopy
8  CFLAGS = -mthumb -mcpu=cortex-m3 -O0 -g
9
10 all: app.bin
11
12 $(BUILD_DIR):
13     mkdir $(BUILD_DIR)
14 crt.o: $(BUILD_DIR) $(SRC_DIR)/crt.s
15     $(AS) -o $(BUILD_DIR)/crt.o $(SRC_DIR)/
   crt.s
16
17 main.o: $(BUILD_DIR) $(SRC_DIR)/main.c
18     $(CC) $(CFLAGS) -c -o $(BUILD_DIR)/main.o
   $(SRC_DIR)/main.c
19
20 app.elf: $(BUILD_DIR) $(SRC_DIR)/linker.ld
   crt.o main.o
21     $(LD) -T $(SRC_DIR)/linker.ld -o $(
   BUILD_DIR)/app.elf \
22     $(BUILD_DIR)/crt.o $(BUILD_DIR)/main.o
23
24 app.bin: app.elf
25     $(BIN) -O binary $(BUILD_DIR)/app.elf $(
   BUILD_DIR)/app.bin
26

```

```

27 clean:
28     rm -rf $(BUILD_DIR)
29
30 openocd-flash: app.bin
31
32     sudo openocd -f interface/stlink.cfg -c "
   set CPUTAPID 0x2ba01477" -f target/
   stm32f1x.cfg \
33     -c "init" -c "reset halt" \
34     -c "flash write_image erase $(BUILD_DIR)/
   app.bin 0x08000000" \
35     -c "reset run" -c "exit"
36 openocd-erase:
37     openocd -f interface/stlink.cfg -f -c "
   set CPUTAPID 0x1ba01477" -f target/
   stm32f1x.cfg \
38     -c "init" -c "reset halt" \
39     -c "flash erase_sector 0 0 last" -c "
   reset run" -c "exit"

```

### III. BLINK LED CON DELAY

Primero se hizo parpadear el LED integrado en la placa (PC13) usando el temporizador de núcleo **SysTick** para generar interrupciones cada 1 ms, lo que permitió una temporización precisa y determinística. Se introdujo una variable global `tick` incrementada por hardware, y la función `delay_ms_bloqueante` utiliza esta variable para crear retardos basados en eventos, representando un avance significativo en la programación de sistemas embebidos. Y por ultimo, se agregó un segundo LED (PA0) que alterna su estado con el LED integrado, demostrando la capacidad de controlar múltiples periféricos de manera eficiente.

El código para hacer parpadear el LED integrado en la placa (PC13) usando el temporizador de núcleo **SysTick** se muestra a continuación:

```

1  #include <stdint.h>
2
3  // register address
4  #define RCC_BASE 0x40021000
5  #define GPIOC_BASE 0x40011000
6
7  #define RCC_APB2ENR *(volatile uint32_t *) (
   RCC_BASE + 0x18)
8
9  #define GPIOC_CRH *(volatile uint32_t *) (
   GPIOC_BASE + 0x04)
10 #define GPIOC_ODR *(volatile uint32_t *) (
   GPIOC_BASE + 0x0C)
11
12 // bit fields
13 #define RCC_IOPCEN (1 << 4)
14 #define RCC_IOPAEN (1 << 2)
15 #define GPIOC13 (1UL << 13)
16
17 // --- Registros del SysTick (Cortex-M Core) ---
18 #define SysTick_BASE 0xE000E010
19 #define SysTick_CTRL *(volatile uint32_t *)
   (SysTick_BASE + 0x00)
20 #define SysTick_LOAD *(volatile uint32_t *)
   (SysTick_BASE + 0x04)
21 #define SysTick_VAL *(volatile uint32_t *)
   (SysTick_BASE + 0x08)
22

```

```

23 // Bits de control de SysTick
24 #define SysTick_CTRL_ENABLE (1 << 0) //
    Habilita el contador
25 #define SysTick_CTRL_TICKINT (1 << 1) //
    Habilita la interrupcion de SysTick
26 #define SysTick_CTRL_CLKSOURCE (1 << 2) //
    Fuente de reloj
27 #define SysTick_CTRL_COUNTFLAG (1 << 16) //
    Bandera de desborde
28
29 volatile uint32_t tick;
30 // funcion que atiende la interrupcion
31 void SysTick_Handler(void)
32 {
33     tick++;
34 }
35
36 // inicializacion del systick
37 void systick_init_ms(void)
38 {
39     tick = 0;
40     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE
    ; // Reloj de SysTick a HCLK/8 (8MHz /
    8 = 1MHz -> 1 tick = 1us)
41     SysTick_LOAD = 999; // Poner valor de
    RECARGA para 1000 ticks (1ms) (1000 -
    1) = 999
42     SysTick_VAL = 0; // reset del contador
43     SysTick_CTRL |= SysTick_CTRL_TICKINT |
    SysTick_CTRL_ENABLE; // Habilitar la
    interrupcion de SysTick (TICKINT) Y el
    contador (ENABLE)
44 }
45
46 // funcion delay
47 void delay_ms_bloqueante(uint32_t ms)
48 {
49     uint32_t tiempo_inicio = tick + ms;
50     // Espera activa hasta que el contador
    global haya avanzado 'ms' milisegundos
51     while (tick != tiempo_inicio) ;
52 }
53
54 void main(void)
55 {
56     RCC_APB2ENR |= RCC_IOPCEN | RCC_IOPAEN;
57
58     GPIOC_CRH &= 0xFF0FFFFFFF;
59     GPIOC_CRH |= 0x00200000;
60
61     systick_init_ms();
62     while (1)
63     {
64         // Encender el LED (poner PC13 en
        BAJO)
65         GPIOC_ODR &= ~GPIOC13;
66         delay_ms_bloqueante(500); // Espera
        500 ms
67         // Apagar el LED (poner PC13 en
        ALTO)
68         GPIOC_ODR |= GPIOC13;
69         delay_ms_bloqueante(500); // Espera
        500 ms
70
71         //
72     }
73 }

```

Listing 2. Implementacion de parpadeo de LEDs con SysTick en main.c

El código para hacer parpadear el LED integrado en la placa (PC13) y el segundo LED (PA0) usando el temporizador de núcleo SysTick se muestra a continuación:

```

1 #include <stdint.h>
2
3 // register address
4 #define RCC_BASE 0x40021000
5 #define GPIOC_BASE 0x40011000
6 #define GPIOA_BASE 0x40010800
7
8 #define RCC_APB2ENR *(volatile uint32_t *) (
    RCC_BASE + 0x18)
9
10 #define GPIOA_CRL *(volatile uint32_t *) (
    GPIOA_BASE + 0x00)
11 #define GPIOA_ODR *(volatile uint32_t *) (
    GPIOA_BASE + 0x0C)
12
13 #define GPIOC_CRH *(volatile uint32_t *) (
    GPIOC_BASE + 0x04)
14 #define GPIOC_ODR *(volatile uint32_t *) (
    GPIOC_BASE + 0x0C)
15
16 // bit fields
17 #define RCC_IOPCEN (1 << 4)
18 #define RCC_IOPAEN (1 << 2)
19 #define GPIOC13 (1UL << 13)
20 #define GPIOA0 (1UL << 0)
21
22 // --- Registros del SysTick (Cortex-M Core) ---
23 #define SysTick_BASE 0xE000E010
24 #define SysTick_CTRL *(volatile uint32_t *)
    (SysTick_BASE + 0x00)
25 #define SysTick_LOAD *(volatile uint32_t *)
    (SysTick_BASE + 0x04)
26 #define SysTick_VAL *(volatile uint32_t *)
    (SysTick_BASE + 0x08)
27
28 // Bits de control de SysTick
29 #define SysTick_CTRL_ENABLE (1 << 0) //
    Habilita el contador
30 #define SysTick_CTRL_TICKINT (1 << 1) //
    Habilita la interrupcion de SysTick
31 #define SysTick_CTRL_CLKSOURCE (1 << 2) //
    Fuente de reloj
32 #define SysTick_CTRL_COUNTFLAG (1 << 16) //
    Bandera de desborde
33
34 volatile uint32_t tick;
35 // funcion que atiende la interrupcion
36 void SysTick_Handler(void)
37 {
38     tick++;
39 }
40
41 // inicializacion del systick
42 void systick_init_ms(void)
43 {
44     tick = 0;
45     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE
    ; // Reloj de SysTick a HCLK/8 (8MHz /
    8 = 1MHz -> 1 tick = 1us)
46     SysTick_LOAD = 999; // Poner valor de
    RECARGA para 1000 ticks (1ms) (1000 -
    1) = 999
47     SysTick_VAL = 0; // reset del contador
48     SysTick_CTRL |= SysTick_CTRL_TICKINT |

```

```

49     SysTick_CTRL_ENABLE; // Habilitar la
        interrupcion de SysTick (TICKINT) Y el
        contador (ENABLE)
50 }
51 // funcion delay
52 void delay_ms_bloqueante(uint32_t ms)
53 {
54     uint32_t tiempo_inicio = tick + ms;
55     // Espera activa hasta que el contador
        global haya avanzado 'ms' milisegundos
56     while (tick != tiempo_inicio) ;
57 }
58
59 void main(void)
60 {
61     RCC_APB2ENR |= RCC_IOPCEN | RCC_IOPAEN;
62
63     GPIOC_CRH &= 0xFF0FFFFFFF;
64     GPIOC_CRH |= 0x00200000;
65
66     GPIOA_CRL &= 0xFFFFFFF0;
67     GPIOA_CRL |= 0x00000003;
68
69     systick_init_ms();
70     while (1)
71     {
72         // Encender el LED (poner PC13 en
        BAJO)
73         GPIOC_ODR &= ~GPIOC13;
74         delay_ms_bloqueante(500); // Espera
        500 ms
75         // Apagar el LED (poner PC13 en
        ALTO)
76         GPIOC_ODR |= GPIOC13;
77         delay_ms_bloqueante(500); // Espera
        500 ms
78         //
79         GPIOA_ODR |= GPIOA0;
80         delay_ms_bloqueante(500);
81         GPIOA_ODR &= ~GPIOA0;
82         delay_ms_bloqueante(500);
83     }
84 }

```

Listing 3. Implementacion de parpadeo de LEDs con SysTick en main.c

El makefile y el linker script se mantuvieron sin cambios, ya que la estructura del proyecto no se modificó significativamente. La principal diferencia radica en la implementación del código en `main.c` para incorporar el uso de SysTick y el control de un segundo LED.

#### IV. DEPURACIÓN

El proceso de depuración se llevó a cabo utilizando **OpenOCD** para la comunicación con el programador **ST-Link V2** y el microcontrolador STM32F103C8T6. Se configuró OpenOCD para establecer una conexión JTAG/SWD, permitiendo la carga del programa y la inspección del estado del microcontrolador en tiempo real. Se utilizaron comandos específicos para iniciar la sesión de depuración, establecer puntos de interrupción, y monitorear las variables y registros relevantes durante la ejecución del programa. Este enfoque facilitó la identificación y corrección de errores, así como la verificación del correcto funcionamiento de los LED y la temporización implementada con SysTick.

Los comandos utilizados para la depuración fueron los que sugiere el trabajo práctico, como `break`, `continue`, `list`, `step`, `next`, `stepi`, y `nexti`. Además, se emplearon las vistas de GDB para mostrar el código fuente, los registros, y el ensamblador, lo que permitió una comprensión del comportamiento del programa en cada etapa de su ejecución.

#### V. CONCLUSIÓN

En este primer trabajo práctico de Técnicas Digitales 2, se logró implementar con éxito la comunicación directa con el hardware del microcontrolador STM32F103C8T6, un flujo de trabajo basado en herramientas GNU ARM Toolchain, el uso de Makefile para la automatización, y la depuración con OpenOCD.

Se pudo modificar los códigos fuente para agregar un led adicional en ambos `main.c` (con y sin SysTick) y se logró hacer parpadear ambos LED de manera eficiente.

#### REFERENCIAS

- [1] DATA-SHEET STM32F103
- [2] Archivo PDF del TP1 Técnicas Digitales 2